

MindWipe: Adaptive Selective Machine Unlearning for Large Language Models

Causally Adaptive Selective Unlearning (CASU) is a proposed machine-unlearning framework designed to remove specific unwanted knowledge from a trained Large Language Model (LLM) while preserving as much of its other knowledge and general capabilities as possible. The project addresses a major limitation of current machine-unlearning approaches: successfully reducing the target knowledge can also damage unrelated knowledge, model utility, or robustness. Rather than simply combining existing methods, CASU assigns a specific role to mechanistic analysis, causal validation, selective parameter modification, and reinforcement learning (RL).

The prototype will use a small open-source LLM, preferably Llama-3.2-1B, so that the complete process can be experimentally analyzed. The main benchmark will be TOFU (Task of Fictitious Unlearning), which provides forget and retain data. A second controlled experiment can use a small dataset created by the researchers themselves. In this controlled setting, the researchers know exactly which information was intentionally introduced into the model, providing stronger evidence for evaluating whether the information can actually be removed.

The experimental pipeline begins with the same trained model and identical forget/retain datasets for every method. First, existing methods such as SimNPO, SIMU, mechanistic unlearning, and an RL-based baseline will be implemented or reproduced. These methods will provide direct comparison points for the proposed approach.

CASU begins by applying mechanistic localization to identify model components—such as particular layers, neurons, or MLP components—that are strongly associated with the unwanted knowledge. However, high attribution alone is not considered sufficient. A causal validation stage temporarily intervenes on candidate components and measures whether the intervention actually reduces the target knowledge while causing minimal damage to retained knowledge. Only candidates that demonstrate useful causal influence are passed to the next stage.

The framework then applies selective modification, inspired by the principle behind SIMU: instead of changing the entire model, only a small, validated subset of parameters or components is modified. This reduces unnecessary changes and potentially improves retention.

An RL controller using PPO is then introduced as an adaptive decision layer. The RL agent does not search the entire billion-parameter model. Instead, it chooses among validated interventions, such as keep, suppress, or modify. Its reward balances four objectives: successful forgetting, preservation of retained knowledge, general model utility, and low modification/collateral damage.

After unlearning, the resulting model is evaluated using forget quality, retain quality, general utility, membership-inference attacks, extraction tests, parameter-change percentage, computational cost, and relearning resistance. Paraphrased and indirect queries will also be used to determine whether the model has genuinely reduced the target knowledge rather than merely learning to refuse a particular question.

The technical stack will primarily consist of Python 3.11, PyTorch, Hugging Face Transformers, Hugging Face Datasets, OpenUnlearning, Accelerate, TRL/PPO, and Streamlit for the experimental dashboard. A CUDA-enabled NVIDIA GPU will be used for practical training and experimentation.

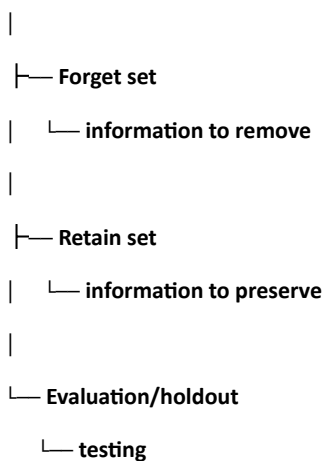


1. Dataset

For the first experiment, use TOFU.

OpenUnlearning already supports TOFU and provides Llama-3.2 1B/3B and other model checkpoints.

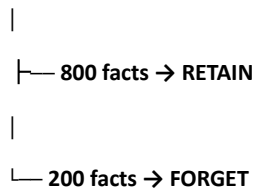
TOFU



For your strongest proof

Also create a controlled dataset:

Controlled Dataset



Train a small model/checkpoint on that dataset.

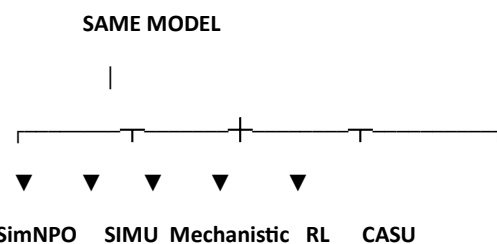
Then you know exactly what information you deliberately taught the model.

That gives you two experiments:

Experiment	Model	Dataset	Purpose
Controlled	Small model/checkpoint	Your known dataset	Prove the mechanism
Benchmark	Llama-3.2-1B	TOFU	Compare against published methods

2. Baselines

Run everything on the same starting checkpoint.



This is critical.

Otherwise the comparison isn't scientifically fair.

SIMU is specifically designed to identify critical neurons and selectively update them rather than broadly modifying the model.

Mechanistic Unlearning instead uses mechanistic localization to identify components associated with the mechanism producing the knowledge; its published results report improved robustness and resistance to relearning in its studied settings.

3. Algorithm inside CASU

Step A — Mechanistic localization

Input:

Forget examples

Run them through the model and inspect:

Layer

↓

Attention

↓

MLP



Neuron/component



Activation

Produce:

Candidate 1

Candidate 2

Candidate 3

...

Candidate 20

We're not changing the model yet.

4. Causal validation

This is the important improvement.

For each candidate:

Temporarily intervene



Run forget queries



Measure change



Run retain queries



Measure collateral damage

Example:

Component A

Forget effect = HIGH

Retain damage = LOW

→ KEEP AS CANDIDATE

But:

Component B

Forget effect = HIGH

Retain damage = HIGH

→ REJECT

So the RL agent doesn't receive thousands of arbitrary parameters.

It receives a small set of experimentally validated actions.

That is the main architectural idea.

5. Selective controller

This is where we borrow the strongest principle from SIMU.

Don't update the whole LLM.

1,000,000,000 parameters

↓

only selected components

↓

small fraction modified

SIMU's central motivation is exactly this: selective updating can preserve more of the model's original capability than broad unlearning updates.

6. RL controller

Now RL enters.

The RL agent sees:

STATE

forget score

retain score

causal effect

collateral effect

layer

component

previous actions

number of modified parameters

Actions

Keep it simple for V1:

0 = KEEP

1 = SUPPRESS

2 = MODIFY

Don't start with complicated RL.

Use PPO initially because it is well understood and available in standard RL libraries.

7. Reward

The reward should NOT simply be:

Forget = +1

Otherwise the model may learn:

"Refuse everything."

Instead:

Reward =

Forgetting

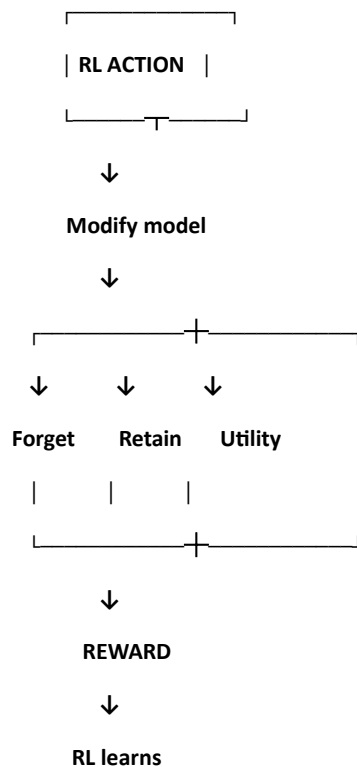
+ Retention

+ General utility

- Collateral damage

- Modification cost

Conceptually:



8. Actual unlearning

After the RL agent chooses an intervention:

Selected component



Targeted update



Model weights change



Save checkpoint

This is the point where the "erasing" happens.

Not literally deleting a file.

You're changing the model parameters so that the target information is no longer represented/expressed in the same way.

9. Verification

This should be a separate module.

Forget test

Original answer



After unlearning



Target knowledge suppressed?

Retain test

Unrelated knowledge



Still works?

Robustness test

Try:

Original question

Paraphrased question

Indirect question

Related question

Multi-hop question

Relearning test

This is particularly important because mechanistic unlearning research has shown that resistance to relearning is a useful robustness criterion.

Unlearn



Fine-tune briefly on forgotten information



Does it come back?

10. Final comparison

Your experiment becomes:

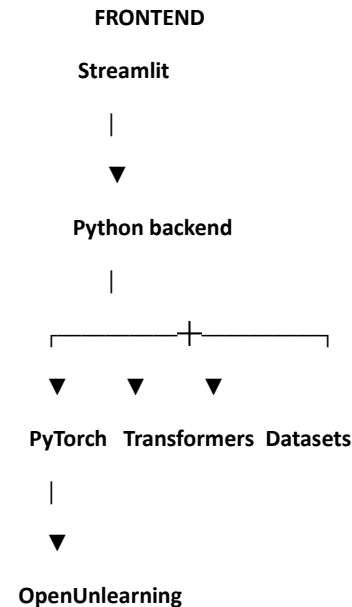
Method	Forget	Retain	Utility	MIA	Relearn	Parameters changed	Time
SimNPO	measure	measure	measure	measure	measure	measure	measure
SIMU	measure	measure	measure	measure	measure	measure	measure
Mechanistic	measure	measure	measure	measure	measure	measure	measure
RL	measure	measure	measure	measure	measure	measure	measure
CASU	measure	measure	measure	measure	measure	measure	measure

Do not put predicted numbers here. These are experimental results.

OpenUnlearning already exposes several relevant evaluation metrics, including forget quality, model utility, extraction strength, exact memorization and MIA metrics, which makes it a good foundation for this comparison.

Technical stack

Keep the stack boring and reliable.



|
└─ SimNPO
└─ NPO
└─ evaluation

|
▼

Mechanistic module

|
▼

CASU algorithm

|
└─ selective masking
└─ causal intervention
└─ PPO controller

|
▼

Evaluation

Software

- Python 3.11
- PyTorch
- Hugging Face Transformers
- Hugging Face Datasets
- OpenUnlearning
- Accelerate
- TRL for PPO/RL experimentation
- Streamlit for dashboard
- Weights & Biases optionally for experiment tracking

OpenUnlearning's current setup itself targets Python 3.11 and supports the Llama/TOFU experiment path you're planning.

Hardware

For the prototype:

Minimum sensible target

NVIDIA GPU

12–24 GB VRAM

A cloud GPU is fine.

Your current PC has Intel UHD graphics and CPU-only PyTorch, so don't waste time trying to force this experiment onto that machine.

EXAMPLE:

Simplest example: A company asks the LLM to forget one employee's information

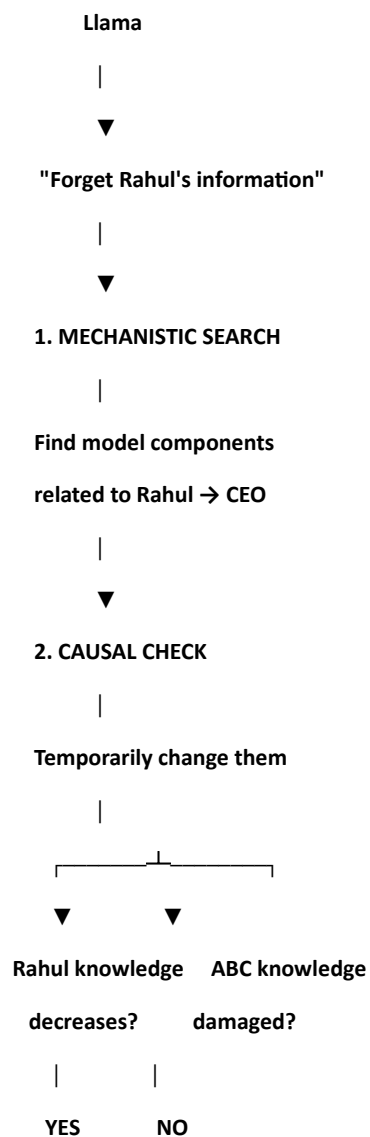
Imagine our small Llama model has learned:

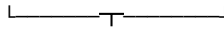
Forget: "Rahul is the CEO of ABC Company."

Retain: "ABC Company was founded in 2010."

Now we want the model to forget Rahul's CEO information without damaging everything else it knows.

How our CASU model works





3. SELECTIVE UPDATE



Change **ONLY** those
relevant components



4. RL CONTROLLER



"Which intervention
causes least damage?"



FINAL LLM

Before unlearning

Question: Who is the CEO of ABC Company?

Model: Rahul.

Question: When was ABC Company founded?

Model: 2010.

After CASU unlearning

Question: Who is the CEO of ABC Company?

Model: I don't know / cannot provide that information.

Question: When was ABC Company founded?

Model: 2010.

The important part

We didn't erase the whole model.

We tried to find the small part of the model responsible for the unwanted information, verified that changing it actually affects Rahul's information, and then let the RL controller choose the least damaging intervention.

So the goal is:

Forget Rahul, but don't forget ABC Company.

That's the simplest real-life analogy for what our proposed model is trying to achieve.

1. Algorithm we are using

Our proposed pipeline is:

Mechanistic Localization → Causal Validation → Selective Unlearning → RL Controller → Verification

Step 1 — Mechanistic Localization

Find which layers/neurons/components are strongly involved in producing the knowledge we want to forget.

Input: Forget examples

Output: Small list of candidate components

Forget data



Model activation analysis



Important components

Step 2 — Causal Validation

We don't trust importance scores alone.

Temporarily intervene on each candidate and check:

- Does the unwanted knowledge decrease?
- Does retained knowledge remain intact?

Only components that actually cause the desired change are selected.

Candidate



Intervene



Forget improves? — No → Reject



Yes



Retain damaged? — Yes → Reject



No



Keep candidate

Step 3 — Selective Unlearning

Instead of changing the entire LLM, modify only the validated components.

This is the selective-update principle we take from SIMU.

1B-parameter model



Selected components



Only those are updated

Step 4 — RL Controller

We use PPO (Proximal Policy Optimization) as the initial RL algorithm.

The RL agent decides which validated intervention is best:

KEEP

MODIFY

SUPPRESS

It receives a reward based on:

good forgetting + good retention – collateral damage – unnecessary modification

So RL is not searching through the entire LLM. It chooses among already validated interventions.

Step 5 — Verification

Finally we test:

Did target knowledge disappear?

Did retained knowledge survive?

Did general capability survive?

Can the forgotten information be extracted?

Can it be recovered through paraphrasing?

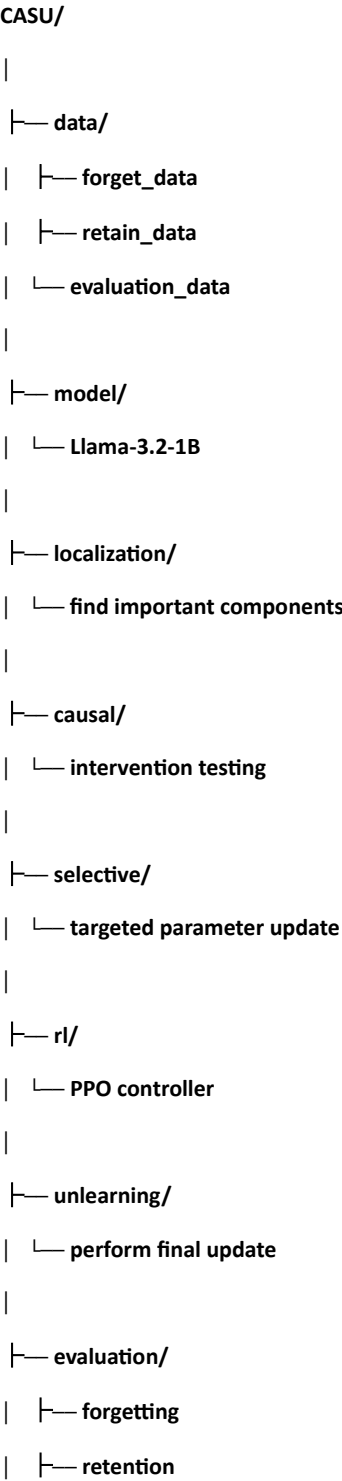
Can it be relearned easily?

2. Technical stack

Layer	Technology
Programming	Python 3.11
Deep learning	PyTorch
LLM	Llama-3.2-1B initially
Model handling	Hugging Face Transformers
Dataset	Hugging Face Datasets
Unlearning framework	OpenUnlearning
RL	PPO / TRL
GPU acceleration	CUDA + NVIDIA GPU

Layer	Technology
Experiment tracking	Weights & Biases (optional)
Dashboard	Streamlit

3. Main software modules



- | └─ MIA
- | └─ relearning
- |
- └─ dashboard/

1. What is the input?

Our system takes three main things:

INPUT

- |
- | └─ 1. Trained LLM
- | └─ Llama-3.2-1B
- |
- | └─ 2. Forget Set
- | └─ information the model must forget
- |
- └─ 3. Retain Set
- └─ information the model must keep

2. What is the Forget Set?

The forget set contains examples representing the knowledge we want to remove.

For example:

Forget Set

Q: Who is the CEO of ABC company?

A: Rahul Sharma

Q: When did Rahul Sharma become CEO?

A: 2018

Q: What position does Rahul Sharma hold?

A: CEO of ABC

We're basically telling the unlearning algorithm:

"This knowledge is the target. Make the model forget it."

3. What is the Retain Set?

The retain set contains knowledge that the model must not lose.

For example:

Retain Set

Q: When was ABC company founded?

A: 2010

Q: Where is ABC company headquartered?

A: Mumbai

Q: What industry does ABC company operate in?

A: Software

We're telling the algorithm:

"Don't damage this knowledge."

4. How do we actually input them?

We don't manually type every question into the model.

We store them as a dataset, usually something like:

dataset/

|

├— forget.json

└— retain.json

For example, forget.json:

```
[
  {
    "question": "Who is the CEO of ABC?",
    "answer": "Rahul Sharma"
  },
  {
    "question": "When did Rahul become CEO?",
    "answer": "2018"
  }
]
```

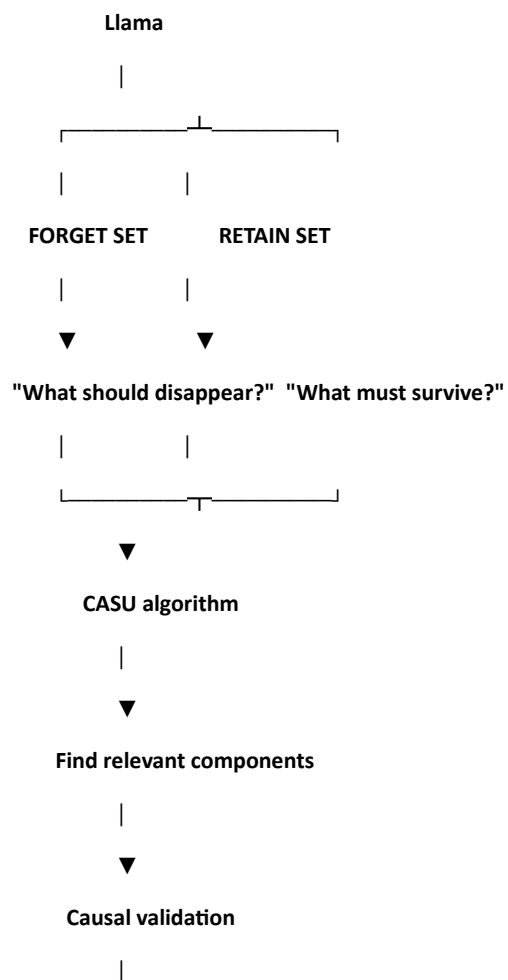

And retain.json:

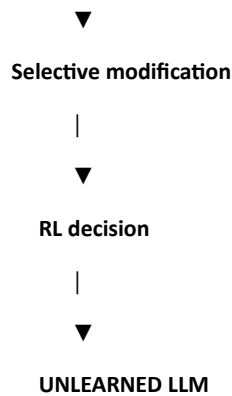
```
[
  {
    "question": "When was ABC founded?",
    "answer": "2010"
  },
  {
    "question": "Where is ABC headquartered?",
    "answer": "Mumbai"
  }
]
```

The Python/Hugging Face dataset loader reads these files and gives the examples to the unlearning algorithm.

5. What happens after input?

Very simply:





6. What is the output?

There are two important outputs.

Output 1 — The new model

Original Llama



Unlearning



Llama-CASU

The new checkpoint should have reduced knowledge of the forget set while preserving the retain set.

Output 2 — Evidence/results

The system also produces:

Forget score → Did it forget?

Retain score → Did it preserve other knowledge?

Utility → Is the model still useful?

MIA score → Can forgotten data still be detected?

Relearning score → Can the forgotten knowledge easily return?

Parameters changed → How much of the model changed?

Training time → How expensive was it?

The easiest way to remember it

Think of the model as a student:

Forget Set:

"Forget Chapter 5."

Retain Set:

"Keep Chapters 1–4 and 6–10."

Unlearning algorithm:

"Find the parts of your brain responsible for Chapter 5 and remove that knowledge without damaging the other chapters."

Output:

Student no longer answers Chapter 5 questions correctly, but still answers the other chapters correctly.

That's exactly what we're trying to demonstrate experimentally with the LLM.